

Aprendendo Física com o Visual Basic

Mário Leite

Primeiramente gostaria que os leitores me desculpassem pelo título, um tanto quanto pretensioso; mas na verdade, é apenas uma “deixa” para mostrar que o Visual Basic (VB) pode auxiliar no estudo das Ciências; em particular a Física. Nesta postagem, preparei um assunto que é a paixão de quase todos os estudantes de Física, ou dos que simplesmente gostam dela: o chamado **MUV** (**M**ovimento **U**niformemente **V**ariado). Conforme sabemos, nesse tipo de movimento o módulo da velocidade instantânea do corpo varia com o tempo (aumenta ou diminui) de um mesmo valor o tempo todo; isto é, sua aceleração é constante. Um caso particular desse tipo de movimento é a **Queda Livre** de um Corpo, sob a aceleração da gravidade, com módulo constante de aproximadamente 9.81 m/s^2 ; isto é, sua velocidade aumenta de 9.81 m/s em cada segundo, à medida que ele cai.

O vetor peso do corpo (a força com que a Terra o atrai – $\vec{P}$), é definido como sendo o produto de sua massa (m) pelo vetor aceleração da gravidade ($\vec{g}$); assim sendo, o módulo dessa força pode ser definido como sendo: $\vec{P} = m * \vec{g}$. Por outro lado, sabemos, também, que quando um corpo está à uma certa altura do solo, tem uma energia armazenada - Energia Potencial – (capacidade de realizar um trabalho), e à medida que ele “cai”, essa energia vai se transformando em Energia Cinética (energia de movimento). Para o nosso aplicativo, vamos considerar o conjunto de medidas adotado pelo sistema **MKSC** do seguinte modo:

- P** = módulo peso do corpo (em Newtons)
- m** = massa do corpo (em Kg)
- H** = distância do corpo até o solo (em metros)
- g** = módulo da aceleração da gravidade (9.81 m/s^2 - valor médio constante ao nível do mar)
- Vo** = módulo da velocidade inicial do corpo - no caso $V_0 = 0$, pois ele é abandonado – (em m/s)
- V** = módulo da velocidade instantânea do corpo na queda (em m/s)
- T** = tempo de queda (em segundos)
- Ep** = energia potencial (em joules)
- Ec** = energia cinética (em joules)

Considerando esses dados, vamos fazer uma pequena dedução para chegarmos à nossa equação de movimento (que é o que nos interessa) a partir da Lei de Conservação da Energia. A Energia Potencial de um corpo pode ser dada por: $E_p = P * H = m * g * H$; e como essa energia se transforma quase totalmente em Energia Cinética [$E_c = (1/2) * m * V^2$] ao tocar o solo, podemos igualar essa energia à equação da Energia Potencial [$E_p = m * g * H = E_c = (1/2) * m * V^2$]. Por outro lado, o módulo da Velocidade (**V**) do corpo pode ser dado pelo produto do módulo da aceleração da gravidade pelo tempo: $V = g * T$; sendo assim, podemos igualar os termos acima da seguinte maneira: $(1/2) * m * V^2 = m * g * H$, e obtemos: $(1/2) * m * (g * T)^2 = m * g * H$.

Fazendo as simplificações necessárias, chegamos finalmente a: $H = (1/2) * g * T^2$. que é a equação que rege o movimento de um corpo em Queda Livre, abandonado a uma distância **H** do solo. É claro que a dedução acima está bem simplificada; a rigor deveríamos usar cálculos mais avançados, como por exemplo, partir da definição de Velocidade Instantânea em seguida integrá-la no tempo para encontrar a equação do espaço percorrido; mas para os nossos propósitos, não há necessidade de todo esse rigor matemático.

Analisando a equação (e relembando o 3º ano Científico – agora é 3º ano do Curso Médio) pode-se verificar que ela representa uma parábola do 2º grau com a concavidade para cima; isto é, a equação de movimento do corpo é do tipo $Y = kX^2$ (onde **k** é uma constante positiva) que no caso é $(1/2) * g$.

Bem, depois de toda essa formalidade, vamos ao que interessa: como usar o VB para ver tudo isso em ação!

O que pretendemos é mostrar o corpo em Queda Livre, e ao mesmo tempo como um observador no solo vê seu movimento; e também os parâmetros mais importantes desse movimento: tempo de queda, velocidade instantânea e o espaço percorrido. A **figura 1** mostra o nosso aplicativo em ação, e a figura 2 apresenta a codificação da procedure-evento **CdmIniciar_Click()**, assim também como a procedure-evento **CdmFinalizar_Click()** que finaliza o aplicativo ao clicar no botão **CmdFinalizar**. Para dar partida no aplicativo, basta um *click* sobre o botão **CmdIniciar** para que possamos observar a queda do corpo. No canto esquerdo do formulário observamos a parábola descrita pelo corpo (vista pelo observador terrestre); mais ao centro vemos o corpo em queda (visto por alguém que abandonou o corpo); e no canto superior direito podemos analisar os parâmetros desse movimento: tempo de queda, velocidade em cada instante e espaço percorrido pelo corpo. Na codificação tivemos que fazer alguns ajustes (não na equação de movimento) para que toda a ação pudesse ser mostrada dentro da configuração de vídeo usada (640*480*16 bits), e o tipo de processador. Desse modo tivemos que considerar alguns dados: tempo de queda de no máximo 3.41 s (com incremento de apenas **0.0097 s**); usamos a parcela **22** (no termo **Corpo.Top+22**) para sincronizar a descida do corpo com o traçado da parábola; também com o *loop For k=1 To 70000...Next k* apenas para “frear” o movimento de descida do corpo (o que poderia também, ser feito com uma rotina específica), de maneira que pudéssemos observá-lo mais suavemente; caso contrário, se não fosse usado esse artifício (ou outro qualquer) a queda seria muito brusca e não daria para notar o movimento; portanto, se for utilizado outro *hardware* possivelmente esses dados terão que ser redimensionados.

De qualquer forma, com esse simples aplicativo pudemos comprovar entre outras coisas, que a velocidade instantânea do corpo aumenta durante a queda, independentemente de sua massa; o que nos leva a entender por que dois corpos, de massas diferentes, abandonados de uma mesma altura, chegam ao solo ao mesmo tempo (claro, desconsiderando a resistência do ar). E com relação ao tipo de curva descrita pelo movimento, dissemos anteriormente que a equação representava uma parábola com concavidade para cima; entretanto, notamos (pela **figura 2**) que sua concavidade é voltada para baixo! Na realidade não existe nenhuma incoerência entre a dedução matemática e o fenômeno físico; o que acontece é que o sistema de coordenadas usado pelo VB adota como origem do eixo Y (tratada por nós como H) a linha superior do vídeo, aumentando no sentido de cima para baixo; por isso a concavidade da curva é voltada para baixo. Aliás é essa a trajetória do corpo que nosso observador realmente vê, considerando a Terra como o seu referencial.

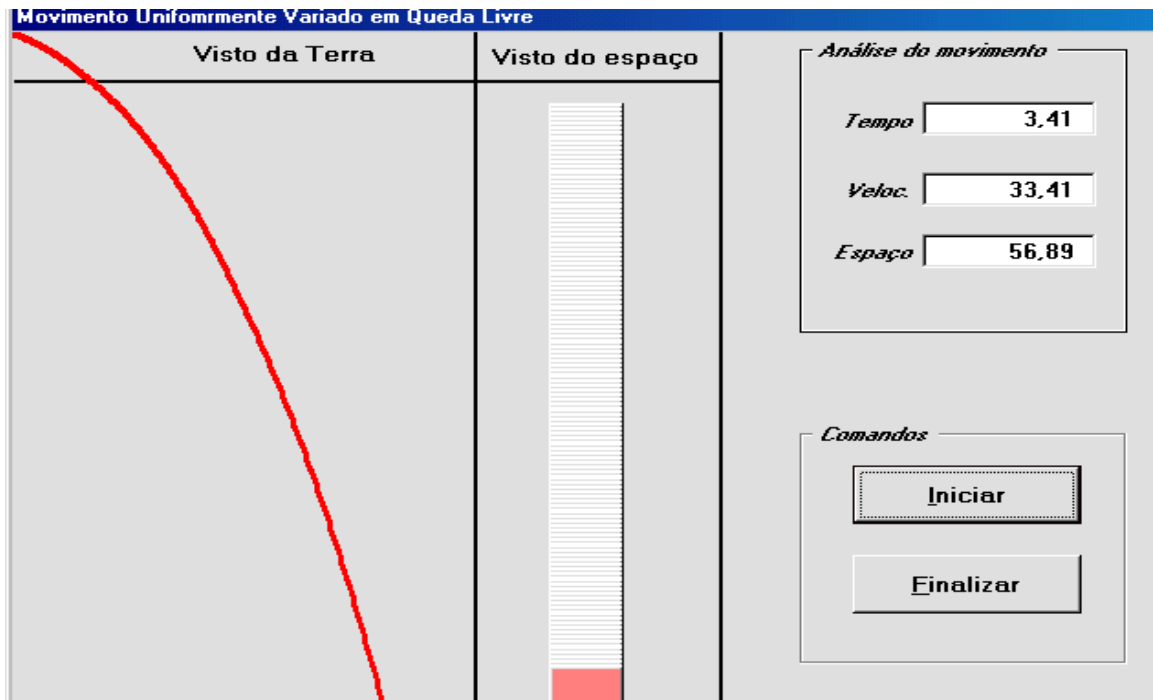


Figura 1 - O movimento do corpo em Queda Livre

```

Private Sub CdmIniciar_Click()
    Dim T, H, V As Single
    ScaleMode = 1 'Configura escala para pixels.
    DrawWidth = 3 'Configura espessura do gráfico.
    X = Corpo.Left
    g = 9.81
    For T = 1 To 3.41 Step 0.0097
        Corpo.Move X, (Corpo.Top + 22)
        H = (1 / 2) * (g * (T ^ 2))
        For k = 1 To 70000
            Next k
            'Traça a trajetória do corpo na cor vermelha
            FrmQuedaLiv.PSet Step(T + 10, H), vbRed
            deltaT = Int(T * 10 ^ 2 + 0.5) / 10 ^ 2
            deltaH = Int(H * 10 ^ 2 + 0.5) / 10 ^ 2
            velocInst = Int((g * T) * 10 ^ 2 + 0.5) / 10 ^ 2
            TxtTempo.Text = "          " & deltaT
            TxtTempo.Refresh
            TxtVeloc.Text = "          " & velocInst
            TxtVeloc.Refresh
            TxtEsp.Text = "          " & deltaH
            TxtEsp.Refresh
        Next T
    End Sub

Private Sub CmdFinalizar_Click()
    'Finaliza o programa
    End
End Sub

```

Figura 2 - O código do aplicativo em VB